

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT WITH COMPOSE

Oleh:
Nazla Salsabila NIM. 2410817320001

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nazla Salsabila
NIM : 2410817320001

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
SOAL 1	6
A. Source Code	8
B. Output Program.....	19
C. Pembahasan.....	22
SOAL 2	26
A. Pembahasan.....	26
SOAL 3	28
A. Pembahasan.....	28
SOAL 4	29
A. Pembahasan.....	29
DAFTAR PUSTAKA	30
TAUTAN GIT.....	31

DAFTAR GAMBAR

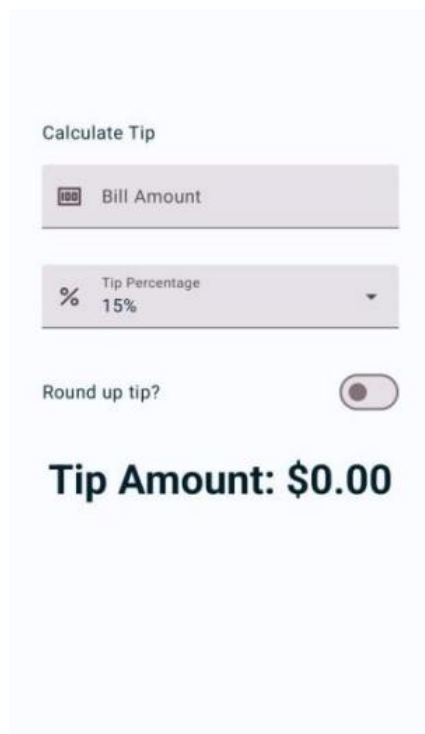
Gambar 1. Tampilan Awal Aplikasi	6
Gambar 2. Tampilan Pilihan Persentase Tip	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan.....	7
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose	19
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose	19
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose	20
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML	20
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML	21
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML	21

DAFTAR TABEL

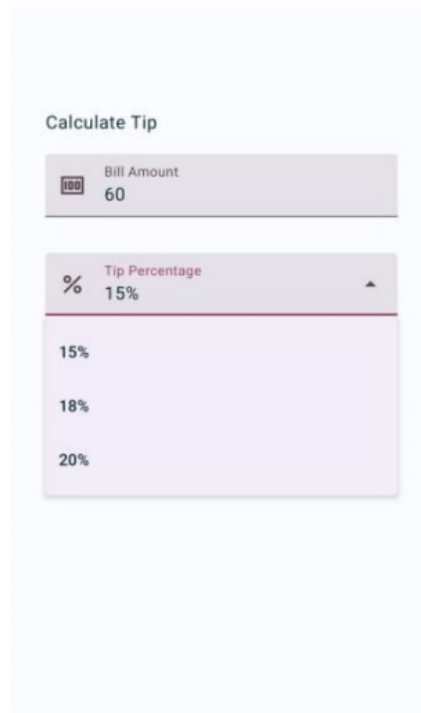
Tabel 1. Source Code MainActivity.kt XML Soal 1	8
Tabel 2. Source Code MainActivity.kt Compose Soal 1	10
Tabel 3. Source Code Activity_Main.XML Soal 1.....	15
Tabel 4. Source Test Case Fitur Function Soal 4.....	29

SOAL 1

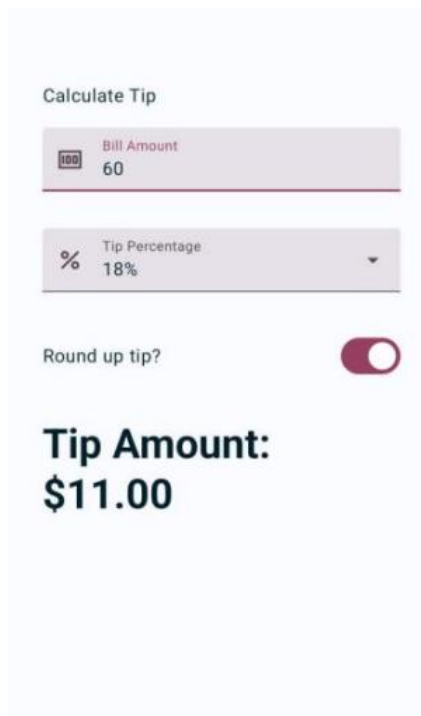
1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:
 - a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
 - b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
 - c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
 - d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt XML

Tabel 1. Source Code MainActivity.kt XML Soal 1

1	package com.nazlasalsabila.tipkalkulator
2	
3	import android.os.Bundle
4	import android.text.Editable
5	import android.text.TextWatcher
6	import android.view.View
7	import android.widget.*
8	import androidx.appcompat.app.AppCompatActivity
9	import androidx.appcompat.widget.SwitchCompat
10	import kotlin.math.ceil
11	
12	class MainActivity : AppCompatActivity() {
13	
14	private lateinit var etBillAmount: EditText
15	private lateinit var spTipPercentage: Spinner
16	private lateinit var switchRound: SwitchCompat
17	private lateinit var tvTipAmount: TextView
18	
19	private val tipValues = arrayOf(0.15, 0.18, 0.20)
20	
21	override fun onCreate(savedInstanceState:
22	Bundle?) {
23	super.onCreate(savedInstanceState)
24	setContentView(R.layout.activity_main)
25	
26	etBillAmount =
27	findViewById(R.id.etBillAmount)
28	spTipPercentage =
29	findViewById(R.id.spTipPercentage)
30	switchRound = findViewById(R.id.switchRound)
31	tvTipAmount = findViewById(R.id.tvTipAmount)
32	
33	val tipLabels = arrayOf("15%", "18%", "20%")
34	val adapter = ArrayAdapter(this,
	android.R.layout.simple_spinner_item, tipLabels)
	adapter.setDropDownViewResource(android.R.layout.simp
	le_spinner_dropdown_item)
	spTipPercentage.adapter = adapter


```

35         spTipPercentage.onItemSelectedListener =
object : AdapterView.OnItemSelectedListener {
36             override fun onItemSelected(parent:
AdapterView<*>?, view: View?, position: Int, id:
Long) {
37                 calculateTip()
38             }
39             override fun onNothingSelected(parent:
AdapterView<*>?) {}
40         }
41
42         switchRound.setOnCheckedChangeListener { _, _
->
43             calculateTip()
44         }
45
46         etBillAmount.addTextChangedListener(object :
TextWatcher {
47             override fun afterTextChanged(s:
Editable?) {
48                 calculateTip()
49             }
50             override fun beforeTextChanged(s:
CharSequence?, start: Int, count: Int, after: Int) {}
51             override fun onTextChanged(s:
CharSequence?, start: Int, before: Int, count: Int)
{}
52         })
53     }
54
55     private fun calculateTip() {
56         val amount =
etBillAmount.text.toString().toDoubleOrNull() ?: 0.0
57         val percent =
tipValues[spTipPercentage.selectedItemPosition]
58         var tip = amount * percent
59
60         if (switchRound.isChecked) {
61             tip = ceil(tip)
62         }
63
64         tvTipAmount.text = "Tip Amount: Rp
%.2f".format(tip)
65     }
66 }

```

MainActivity.kt Compose

Tabel 2. Source Code MainActivity.kt Compose Soal 1

1	package com.nazlasalsabila.tipkalkulator
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.background
7	import androidx.compose.foundation.border
8	import androidx.compose.foundation.layout.*
9	import
	androidx.compose.foundation.shape.RoundedCornerShape
10	import
	androidx.compose.foundation.text.KeyboardOptions
11	import androidx.compose.material3.*
12	import androidx.compose.runtime.*
13	import androidx.compose.ui.Alignment
14	import androidx.compose.ui.Modifier
15	import androidx.compose.ui.graphics.Color
16	import androidx.compose.ui.text.TextStyle
17	import androidx.compose.ui.text.font.FontWeight
18	import androidx.compose.ui.text.input.KeyboardType
19	import androidx.compose.ui.text.style.TextAlign
20	import androidx.compose.ui.unit.dp
21	import androidx.compose.ui.unit.sp
22	import kotlin.math.ceil
23	
24	class MainActivityCompose : ComponentActivity() {
25	override fun onCreate(savedInstanceState:
	Bundle?) {
26	super.onCreate(savedInstanceState)
27	setContent {
28	TipApp()
29	}
30	}
31	}
32	
33	@OptIn(ExperimentalMaterial3Api::class)
34	@Composable
35	fun TipApp() {
36	var bill by remember { mutableStateOf("") }
37	var expanded by remember { mutableStateOf(false)
	}

```

38     val tips = listOf(15, 18, 20)
39     var selectedTip by remember {
mutableStateOf(tips[0]) }
40     var roundUp by remember { mutableStateOf(false) }
41
42     val amount = bill.toDoubleOrNull() ?: 0.0
43     var tipResult = amount * selectedTip / 100
44     if (roundUp) tipResult = ceil(tipResult)
45
46     val purpleNazla = Color(0xFF7B1FA2)
47     val customBrown = Color(0xFF5D4037)
48     val darkerGrayBg = Color(0xFFE0E0E0)
49     val softBlack = Color(0xFF333333)
50
51     Column(
52         modifier = Modifier
53             .fillMaxSize()
54             .background(Color(0xFFFFAFAFA))
55             .padding(20.dp)
56     ) {
57         Text(
58             text = "Calculate Tip",
59             fontSize = 20.sp,
60             fontWeight = FontWeight.Bold,
61             color = softBlack
62         )
63
64         Spacer(Modifier.height(32.dp))
65
66         OutlinedTextField(
67             value = bill,
68             onValueChange = { bill = it },
69             label = {
70                 Text(
71                     "Bill Amount",
72                     fontWeight = FontWeight.Bold,
73                     modifier = Modifier.padding(top =
4.dp, start = 4.dp)
74                 )
75             },
76             leadingIcon = {
77                 Box(
78                     modifier = Modifier
79                         .padding(start = 12.dp)
80                         .size(24.dp)

```

```

81         .border(1.5.dp, customBrown,
RoundedCornerShape(4.dp)),
82         contentAlignment =
Alignment.Center
83     ) {
84         Text("100", fontSize = 10.sp,
color = customBrown, fontWeight = FontWeight.Bold)
85     }
86 },
87     modifier = Modifier
88         .fillMaxWidth()
89         .height(70.dp)
90         .background(darkerGrayBg,
RoundedCornerShape(8.dp)),
91     textStyle = TextStyle(fontSize = 18.sp,
fontWeight = FontWeight.Bold),
92     keyboardOptions =
93     KeyboardOptions(keyboardType = KeyboardType.Number),
94     shape = RoundedCornerShape(8.dp),
95     colors =
OutlinedTextFieldDefaults.colors(
96         focusedBorderColor = purpleNazla,
97         unfocusedBorderColor =
Color.Transparent,
98         focusedLabelColor = purpleNazla,
99         unfocusedLabelColor = customBrown,
100        cursorColor = purpleNazla,
101        focusedContainerColor =
Color.Transparent,
102        unfocusedContainerColor =
Color.Transparent
103    )
104 )
105     Spacer(Modifier.height(24.dp))
106
107     ExposedDropDownMenuBox(
108         expanded = expanded,
109         onExpandedChange = { expanded = !expanded
},
110         modifier = Modifier.fillMaxWidth()
111     ) {
112         OutlinedTextField(
113             value = "$selectedTip%",
114             onValueChange = {},

```

```

115         readOnly = true,
116         label = {
117             Text(
118                 "Tip Percentage",
119                 fontWeight = FontWeight.Bold,
120                 modifier =
Modifier.padding(top = 4.dp, start = 4.dp)
121             )
122         },
123         leadingIcon = {
124             Text(
125                 "%",
126                 fontSize = 20.sp,
127                 fontWeight =
FontWeight.ExtraBold,
128                 color = customBrown,
129                 modifier =
Modifier.padding(start = 16.dp)
130             )
131         },
132         trailingIcon = {
ExposedDropDownMenuDefaults.TrailingIcon(expanded =
expanded) },
133         modifier = Modifier
134             .menuAnchor()
135             .fillMaxWidth()
136             .height(70.dp)
137             .background(darkerGrayBg,
RoundedCornerShape(8.dp)),
138         textStyle = TextStyle(
139             fontSize = 18.sp,
140             fontWeight = FontWeight.Bold,
141             textAlign = TextAlign.Start
142         ),
143         shape = RoundedCornerShape(8.dp),
144         colors =
OutlinedTextFieldDefaults.colors(
145             focusedBorderColor = purpleNazla,
146             unfocusedBorderColor =
Color.Transparent,
147             focusedLabelColor = purpleNazla,
148             unfocusedLabelColor =
customBrown,
149             focusedContainerColor =
Color.Transparent,

```

```

150         unfocusedContainerColor =
Color.Transparent
151     )
152 )
153     ExposedDropDownMenu(
153         expanded = expanded,
155         onDismissRequest = { expanded = false
}
156     ) {
157         tips.forEach { tipOption ->
158             DropdownMenuItem(
159                 text = {
160                     Text(
161                         text = "$tipOption%",
162                         color = Color.Black,
163                         fontWeight =
FontWeight.Bold
164                     )
165                 },
166                 onClick = {
167                     selectedTip = tipOption
168                     expanded = false
169                 }
170             )
171         }
172     }
173 }
174
175     Spacer(Modifier.height(24.dp))
176
177     Row(
178         modifier = Modifier.fillMaxWidth(),
179         verticalAlignment =
Alignment.CenterVertically
180     ) {
181         Text(
182             text = "Round up tip?",
183             fontSize = 20.sp,
184             fontWeight = FontWeight.Bold,
185             color = softBlack
186         )
187         Spacer(Modifier.weight(1f))
188         Switch(
189             checked = roundUp,
190             onCheckedChange = { roundUp = it },

```

191	colors = SwitchDefaults.colors(
192	checkedThumbColor = purpleNazla,
193	checkedTrackColor =
	purpleNazla.copy(alpha = 0.5f)
194	)
195	)
196	}
197	
198	Spacer(Modifier.height(48.dp))
199	
200	Text(
201	text = "Tip Amount:
	`\${ "%.2f".format(tipResult) }`,
202	fontSize = 38.sp,
203	fontWeight = FontWeight.Black,
204	color = Color.Black,
205	modifier = Modifier.fillMaxWidth(),
206	textAlign = TextAlign.Center
207	)
208	}
209	}

Activity_Main.xml

Tabel 3. Source Code Activity_Main.XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/and
	roid"
4	xmlns:app="http://schemas.android.com/apk/res-
	auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="20dp"
8	android:background="#FAFAFA">
9	
10	<TextView
11	android:id="@+id/title"
12	android:layout_width="wrap_content"
13	android:layout_height="wrap_content"
14	android:text="Calculate Tip"
	android:textSize="18sp"
15	android:textStyle="bold"
16	android:textColor="#CC000000"
17	app:layout_constraintTop_toTopOf="parent"

```

18
19 app:layout_constraintStart_toStartOf="parent"/>
20
21 <LinearLayout
22     android:id="@+id/billBox"
23     android:layout_width="0dp"
24     android:layout_height="70dp"
25     android:orientation="horizontal"
26     android:gravity="center_vertical"
27     android:padding="15dp"
28     android:background="@drawable/bottom_line"
29 app:layout_constraintTop_toBottomOf="@id/title"
30 app:layout_constraintStart_toStartOf="parent"
31 app:layout_constraintEnd_toEndOf="parent"
32 android:layout_marginTop="16dp">
33
34 <TextView
35     android:layout_width="25dp"
36     android:layout_height="25dp"
37     android:text="100"
38     android:textStyle="bold"
39     android:gravity="center"
40     android:textSize="10sp"
41     android:textColor="#5D4037"
42     android:background="@drawable/box_border"
43     android:layout_marginEnd="10dp"/>
44
45 <EditText
46     android:id="@+id/etBillAmount"
47     android:layout_width="0dp"
48     android:layout_height="wrap_content"
49     android:layout_weight="1"
50     android:hint="Bill Amount"
51     android:inputType="numberDecimal"
52     android:background="@null"
53     android:textColorHint="#5D4037"/>
54 </LinearLayout>
55
56 <LinearLayout
57     android:id="@+id/tipBox"
58     android:layout_width="0dp"
59     android:layout_height="70dp"
60     android:orientation="vertical"
61     android:padding="10dp"
62     android:background="@drawable/bottom_line"

```



```

63 app:layout_constraintTop_toBottomOf="@id/billBox"
64     app:layout_constraintStart_toStartOf="parent"
65     app:layout_constraintEnd_toEndOf="parent"
66     android:layout_marginTop="35dp">
67
68     <TextView
69         android:layout_width="wrap_content"
70         android:layout_height="wrap_content"
71         android:text="Tip Percentage"
72         android:textSize="12sp"
73         android:textColor="#5D4037"
74         android:layout_marginStart="41dp"
75         android:layout_marginBottom="5dp"/>
76
77     <LinearLayout
78         android:layout_width="match_parent"
79         android:layout_height="0dp"
80         android:layout_weight="1"
81         android:orientation="horizontal"
82         android:gravity="center_vertical">
83
84         <TextView
85             android:layout_width="35dp"
86             android:layout_height="match_parent"
87             android:text="%"
88             android:textSize="22sp"
89             android:textStyle="bold"
90             android:textColor="#5D4037"
91             android:gravity="center"/>
92
93         <Spinner
94             android:id="@+id/spTipPercentage"
95             android:layout_width="0dp"
96             android:layout_height="match_parent"
97             android:layout_weight="1"
98             android:popupBackground="#F3E5F5"
99     android:dropDownVerticalOffset="45dp"/>
100     </LinearLayout>
101
102 </LinearLayout>
103
104 <TextView
105     android:id="@+id/txtRound"
106     android:layout_width="0dp"
107     android:layout_height="wrap_content"

```

```

108         android:text="Round up tip?"
109         android:textStyle="bold"
110         android:textSize="16sp"
111         android:textColor="#CC000000"
112         app:layout_constraintTop_toBottomOf="@id/tipBox"
113         app:layout_constraintStart_toStartOf="parent"
114 app:layout_constraintEnd_toStartOf="@id/switchRound"
115         android:layout_marginTop="35dp"/>
116
117     <androidx.appcompat.widget.SwitchCompat
118         android:id="@+id/switchRound"
119         android:layout_width="wrap_content"
120         android:layout_height="wrap_content"
121 app:layout_constraintTop_toTopOf="@id/txtRound"
122 app:layout_constraintBottom_toBottomOf="@id/txtRound"
123         app:layout_constraintEnd_toEndOf="parent"
124         app:thumbTint="@color/switch_thumb"
125         app:trackTint="@color/switch_track"/>
126
127     <TextView
128         android:id="@+id/tvTipAmount"
129         android:layout_width="0dp"
130         android:layout_height="wrap_content"
131         android:text="Tip Amount: $0.00"
132         android:textSize="38sp"
133         android:textStyle="bold"
134         android:textColor="#222222"
135         android:gravity="center"
136 app:layout_constraintTop_toBottomOf="@id/switchRound"
137         app:layout_constraintStart_toStartOf="parent"
138         app:layout_constraintEnd_toEndOf="parent"
139         android:layout_marginTop="35dp"/>
140
141 </androidx.constraintlayout.widget.ConstraintLayout>

```

B. Output Program

Calculate Tip

Bill Amount

Tip Percentage
% 15%

Round up tip? ☐

Tip Amount: \$0,00

Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose

Calculate Tip

Bill Amount
100 60

Tip Percentage
% 15%

15%

18%

20%

Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose

Calculate Tip

Bill Amount

100 60

Tip Percentage

% 18%

Round up tip? ☒

Tip Amount: \$11,00

Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose

Calculate Tip

100 Bill Amount

Tip Percentage

% 15%

Round up tip? ☐

Tip Amount: \$0,00

Gambar 7. Screenshot Hasil Jawaban Soal 1 XML

Calculate Tip

Tip Percentage

% 15%

Round

15%

18%

20%

Gambar 8. Screenshot Hasil Jawaban Soal 1 XML

Calculate Tip

Tip Percentage

% 18%

Round up tip?

Tip Amount: \$11,00

Gambar 9. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

MainActivity.kt XML

Kode ini untuk aplikasi sederhana 'Tip Calculator' yang dibuat menggunakan Kotlin dengan tampilan berbasis XML. Pada 'MainActivity', terdapat beberapa komponen seperti 'EditText' untuk memasukkan jumlah tagihan, 'Spinner' untuk memilih persentase tip, 'SwitchCompat' untuk mengatur pembulatan, dan 'TextView' untuk menampilkan hasil perhitungan tip.

Pada metode 'onCreate()', layout 'activity_main.xml' dipanggil menggunakan 'setContentView()'. Setelah itu, setiap komponen dihubungkan ke kode menggunakan 'findViewById()' sesuai dengan ID yang sudah dibuat pada XML. Data persentase tip kemudian ditampilkan pada 'Spinner' menggunakan 'ArrayAdapter' dengan pilihan 15%, 18%, dan 20%, sehingga pengguna bisa memilih persentase tip.

Untuk interaksi pengguna, 'Spinner' diberi 'onItemSelectedListener' agar setiap perubahan pilihan langsung memanggil fungsi 'calculateTip()'. Selain itu, 'SwitchCompat' digunakan untuk mengatur pembulatan melalui 'setOnCheckedChangeListener'. Input pada 'EditText' juga dipantau menggunakan 'TextWatcher', sehingga setiap kali pengguna mengetik, perhitungan tip akan langsung diperbarui.

Fungsi 'calculateTip()' digunakan untuk menghitung tip berdasarkan input yang diberikan. Nilai tagihan diambil dari 'EditText' lalu diubah menjadi angka menggunakan 'toDoubleOrNull()', dan jika kosong akan dianggap 0. Persentase tip diambil dari pilihan pada 'Spinner', kemudian dihitung dengan mengalikan jumlah tagihan dan persentase tersebut. Jika opsi pembulatan diaktifkan, maka hasil tip akan dibulatkan ke atas menggunakan fungsi 'ceil'.

Hasil akhir ditampilkan pada 'TextView' menggunakan format dua angka di belakang koma dengan '"Rp %.2f".format(tip)', sehingga hasil terlihat lebih rapi.

MainActivity.kt Compose

Kode ini untuk aplikasi sederhana ‘Tip Calculator’ yang dibuat menggunakan Kotlin dengan Jetpack Compose, sehingga tampilan dibuat langsung di dalam kode tanpa menggunakan XML. Pendekatan ini membuat penulisan UI dan logika bisa dilakukan dalam satu tempat. Pada kelas ‘MainActivityCompose’, metode ‘onCreate()’ digunakan untuk menampilkan aplikasi melalui ‘setContent’, kemudian memanggil fungsi ‘TipApp()’ sebagai tampilan utama yang akan ditampilkan ke layar.

Fungsi ‘TipApp()’ berisi tampilan sekaligus logika perhitungan. Di dalamnya terdapat beberapa data seperti ‘bill’ untuk menyimpan input jumlah tagihan, ‘selectedTip’ untuk menyimpan persentase tip yang dipilih, ‘expanded’ untuk mengatur kondisi dropdown terbuka atau tertutup, serta ‘roundUp’ untuk menentukan apakah hasil tip dibulatkan. Nilai yang dimasukkan akan diubah menjadi angka menggunakan ‘toDoubleOrNull()’, lalu dihitung berdasarkan persentase tip yang dipilih. Jika pembulatan diaktifkan, maka hasil tip akan dibulatkan ke atas menggunakan fungsi ‘ceil’.

Tampilan disusun menggunakan ‘Column’ agar semua komponen tersusun ke bawah secara rapi dan terstruktur. Di bagian atas terdapat ‘Text’ sebagai judul aplikasi. Untuk input jumlah tagihan digunakan ‘OutlinedTextField’ yang sudah diatur tampilannya agar lebih menarik, termasuk tambahan ikon kecil di sebelah kiri. Field ini juga dibatasi hanya untuk input angka.

Pemilihan persentase tip dibuat menggunakan ‘ExposedDropdownMenuBox’ yang berisi ‘OutlinedTextField’ dan menu dropdown. Saat bagian ini ditekan, akan muncul pilihan seperti 15%, 18%, dan 20% yang bisa dipilih oleh pengguna. Setelah dipilih, nilai tersebut langsung disimpan ke dalam ‘selectedTip’ dan digunakan dalam perhitungan.

Berikutnya terdapat ‘Row’ yang berisi teks “Round up tip?” dan ‘Switch’ di sampingnya. Switch ini digunakan untuk mengatur apakah hasil tip ingin dibulatkan atau tidak. Ketika

switch diaktifkan atau dimatikan, nilai akan langsung diperbarui karena menggunakan state di Compose.

Hasil tip ditampilkan menggunakan 'Text' dengan ukuran lebih besar dan posisi di tengah agar mudah dilihat oleh pengguna. Nilai ditampilkan dengan dua angka di belakang koma menggunakan `"%.2f".format(result)`, sehingga hasil terlihat lebih rapi dan jelas. Bagian ini menjadi fokus utama karena menunjukkan hasil akhir dari perhitungan yang telah dilakukan.

Activity_main.xml

Kode XML ini digunakan untuk mengatur tampilan aplikasi 'Tip Calculator' dengan 'ConstraintLayout' sebagai layout utama. Layout ini dipilih karena memudahkan pengaturan posisi antar komponen agar tetap rapi tanpa perlu banyak layout tambahan. Penggunaan constraint membuat tampilan lebih fleksibel ketika ukuran layar berbeda. Tampilan juga diberi padding dan warna latar supaya tampilan tidak terlalu penuh.

Di bagian atas terdapat 'TextView' dengan teks "Calculate Tip" sebagai judul aplikasi. Teks dibuat lebih besar dan tebal agar langsung terlihat saat aplikasi dibuka, sehingga pengguna bisa langsung memahami fungsi aplikasi. Posisi judul diatur di bagian atas kiri menggunakan constraint agar tampil konsisten.

Di bawahnya terdapat 'LinearLayout' dengan id 'billBox' yang digunakan sebagai tempat input jumlah tagihan. Di dalamnya terdapat 'TextView' kecil sebagai elemen tambahan visual, serta 'EditText' dengan id 'etBillAmount' untuk memasukkan jumlah uang. Susunan dibuat horizontal agar input terlihat dalam satu baris dan lebih mudah digunakan oleh pengguna.

Selanjutnya terdapat 'LinearLayout' dengan id 'tipBox' untuk memilih persentase tip. Di dalamnya terdapat 'TextView' sebagai label penjelas, lalu di bagian bawahnya terdapat

simbol “%” dan ‘Spinner’. ‘Spinner’ ini akan menampilkan pilihan persentase seperti 15%, 18%, dan 20%, sehingga pengguna dapat memilih sesuai kebutuhan.

Di bagian berikutnya terdapat ‘TextView’ dengan teks “Round up tip?” yang berfungsi sebagai keterangan, serta ‘SwitchCompat’ di sampingnya sebagai tombol untuk mengaktifkan atau menonaktifkan pembulatan tip. Kedua komponen ini disusun sejajar agar mudah dipahami dan tidak membingungkan pengguna saat digunakan.

Terakhir, terdapat ‘TextView’ dengan id ‘tvTipAmount’ yang digunakan untuk menampilkan hasil perhitungan tip. Ukuran teks dibuat lebih besar dan diletakkan di tengah agar hasil terlihat jelas dan mudah dibaca. Bagian ini menjadi fokus utama setelah pengguna memasukkan data.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan implementasi XML dan Jetpack Compose terletak pada cara membuat tampilan aplikasi. Pada XML, tampilan ditulis di file terpisah seperti 'activity_main.xml', sedangkan logika ditulis di Kotlin, sehingga pengembang harus menghubungkan keduanya secara manual. Sedangkan Jetpack Compose, tampilan dan logika ditulis langsung dalam satu file Kotlin menggunakan fungsi seperti '@Composable'. Jadi, XML menggunakan pemisahan antara UI dan logika, sedangkan Compose menggabungkannya dalam satu tempat sehingga lebih praktis saat pengembangan.

Pada implementasi XML, kelebihanannya adalah struktur tampilan lebih jelas karena disusun dalam bentuk layout. Cara ini lebih mudah dipahami, terutama bagi pemula, karena komponen terlihat secara langsung. Selain itu, XML sudah lama digunakan sehingga referensi dan contoh sangat banyak. XML juga stabil dan umum dipakai di berbagai aplikasi. Namun, kekurangannya adalah penulisan kode menjadi lebih panjang karena harus menggunakan 'findViewById()' untuk menghubungkan komponen. Jika tampilan semakin banyak, kode akan terasa lebih rumit dan kurang efisien.

Sedangkan pada Jetpack Compose, kelebihanannya adalah penulisan kode lebih ringkas karena tampilan dan logika berada dalam satu file. Perubahan data juga lebih mudah ditangani karena menggunakan state, sehingga tampilan bisa langsung diperbarui tanpa banyak kode tambahan, sehingga membuat proses pembuatan UI lebih cepat dan fleksibel. Namun, kekurangannya adalah perlu penyesuaian karena konsepnya berbeda dari XML. Selain itu, Compose bisa terasa lebih berat saat dijalankan di Android Studio, terutama jika menggunakan emulator.

XML lebih cocok untuk memahami dasar karena strukturnya jelas dan terpisah, sedangkan Jetpack Compose lebih cocok untuk pengembangan yang lebih modern karena lebih sederhana dan efisien dalam penulisan kode.

SOAL 3

Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**

A. Pembahasan

Kotlin memiliki beberapa fitur yang sering digunakan dalam pengembangan Android karena membantu membuat kode lebih aman dan praktis. Salah satunya adalah *null safety* yang berfungsi mencegah error akibat nilai kosong. Selain itu, ada *extension functions* untuk menambah fungsi tanpa mengubah kelas asli, serta *coroutines* yang memudahkan pengolahan proses secara asynchronous. Kotlin juga menyediakan *data class* yang membuat penulisan kode untuk data menjadi lebih singkat. (Mateus & Martinez, 2020).

Dalam penerapannya, fitur Kotlin juga digunakan dalam sistem pengelolaan data karyawan berbasis cloud. *Null safety* membantu mengurangi kesalahan saat input data, sedangkan *coroutines* mendukung proses data agar bisa berjalan lebih cepat dan real-time. Selain itu, Kotlin bisa digunakan bersama Java sehingga lebih mudah menggabungkan dengan library yang sudah ada. Dengan sintaks yang sederhana, pengembangan aplikasi jadi lebih cepat dan tidak terlalu rumit, sehingga Kotlin cukup efektif digunakan dalam sistem informasi berbasis aplikasi (Nasution, 2025).

SOAL 4

Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Pembahasan

Test case penggunaan fitur ‘function’ pada Kotlin dapat dilihat pada fungsi ‘calculateTip()’ yang digunakan dalam aplikasi Tip Calculator.

Tabel 4. Source Test Case Fitur Function Soal 4

1	private fun calculateTip() {
2	val amount =
	etBillAmount.text.toString().toDoubleOrNull() ?: 0.0
3	val percent =
	tipValues[spTipPercentage.selectedItemPosition]
4	var tip = amount * percent
5	
6	if (switchRound.isChecked) {
7	tip = ceil(tip)
8	}
9	
10	tvTipAmount.text = "Tip Amount: Rp
	%.2f".format(tip)
11	}

Fungsi ‘calculateTip()’ digunakan untuk menghitung jumlah tip berdasarkan input pengguna dan persentase yang dipilih. Fungsi ini dipanggil setiap terjadi perubahan data, seperti saat pengguna mengisi jumlah uang, memilih persentase, atau mengaktifkan pembulatan.

DAFTAR PUSTAKA

- Mateus, B. G., & Martinez, M. (2020, October). On the adoption, usage and evolution of Kotlin features in Android development. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)* (pp. 1-12).
- Nasution, I. S. (2025). Penerapan Kotlin Dalam Sistem Pengumpulan Data Karyawan. *Jurnal Gemilang Informatika (GIT)*, 3(1), 7-12.

TAUTAN GIT

<https://github.com/nazlasalsabilaa/PemrogramanMobile.git>